습관 몬스터 API 명세

Oct 15, 2026 · @윤서 · v4

API 목록 문서의 요청을 HTTP 메서드, 경로, 요청·응답 필드로 풀어 쓴 명세입니다. 필드 이름과 값은 schema.sql v1.6 기준이며, 이번 변경으로 DB·시드에 반영해야 할 것은 맨 끝에 모았습니다. 검토 후 보류한 항목은 별도 문서「API 명세 보류 항목」에 정리했습니다.

변경 요약 (v1 → v4)

규칙

- 업무 날짜 경계를 새벽 4시에서 **00시**로 변경. 주는 월요일 00시, 월은 1일 00시 시작
- 습관 점수는 HTML 공식 **0~25**(1점 = 1G), 전체 성실도는 **목표 달성률** %
- 골드·카테고리 경험치·기간 보너스 모두 **점수 상위 3개 습관**만 반영
- 기간 보너스는 **골드**로 지급. 기간은 **했다고 제출한 날 수** 기준(연속일 필요 없음)
- 카테고리 최대 레벨 7 → **20**, 종 해금 레벨 분산
- 탐색: 가진 종이 등장하면 볼 없이 골드 지급, 안 가진 종 등장 가중치 상향. 포획률은 **희귀도별**. 꽝 확률 **0.35**
- 첫 제출이 전부 "못했어"인 날은 탐색 1회
- 이메일 하나에 계정 하나. 다른 가입 방법으로 가입된 이메일은 자동 병합하지 않고 안내
- Apple 로그인 제외

API

- 습관 API 4개(목록·추가·연장·그만두기), 남은 등장 조회 1개 추가 → 44개
- 쓰기 요청에 `businessDate`를 넣어 날짜가 바뀐 뒤 들어온 요청을 거절
- 중복 요청 방지 키 규칙과 요청 검사 순서 명시, 목표 완료·카테고리 열기에 키 추가
- 에러 응답에 선택 필드 `errors`, 에러 코드 전체 목록 추가
- PATCH 규칙: 필드를 안 보내면 변경 없음, `null`이면 삭제

v4 수정

- 요청 검사 순서에 온보딩 여부 검사(②) 추가
- 달성률은 반올림한 점수로 계산함을 명시
- 통계 예시의 카테고리 경험치를 체크인 예시와 맞춤
- `explore_miss_rate` 0.35로 확정(베타에서 튜닝)

공통 규칙

- **기본 경로:** `/api/v1`. 아래 경로는 모두 이 뒤에 붙습니다.
- **인증:** `/auth/*`를 뺀 모든 요청은 헤더 `Authorization: Bearer {accessToken}`이 필요합니다.
  - 토큰이 없거나 형식이 틀리거나 폐기되었으면 401 `INVALID_TOKEN`, 만료되었으면 401 `TOKEN_EXPIRED`. 앱은 `TOKEN_EXPIRED`일 때만 재발급을 시도합니다.

    - **재발급은 앱 전체에서 한 번에 하나만** 진행합니다. 요청 여러 개가 동시에 `TOKEN_EXPIRED`를 받으면, 첫 요청만 `/auth/refresh`를 부르고 나머지는 그 결과를 기다렸다가 새 토큰으로 다시 보냅니다. 리프레시 토큰은 매번 교체되므로, 동시에 재발급하면 두 번째 요청이 폐기된 토큰을 쓰게 되어 로그아웃됩니다.
    - 온보딩을 마치지 않은 사용자가 온보딩 관련 요청(`/categories`, `/measure-types`, `/onboarding`) 외의 요청을 하면 403 `ONBOARDING_REQUIRED`.
- **날짜**: 업무 날짜는 `"2026-10-12"` 형식이며 **00시(Asia/Seoul) 경계**로 서버가 계산합니다. 시각은 `"2026-10-12T14:35:00+09:00"` 형식입니다.
- **업무 날짜 검사**: 체크인 제출·수정, 탐색, 포획 요청은 본문에 `businessDate`(앱이 화면을 연 날짜)를 담습니다. 서버의 오늘 날짜와 다르면 409 `BUSINESS_DATE_CHANGED`를 돌려주고, 앱은 화면을 새로 불러옵니다.
- **중복 요청 방지 키**: 헤더 `Idempotency-Key: {UUID}`. 클라이언트가 버튼을 누를 때 새로 만들고, 재전송할 때는 같은 값을 보냅니다.
    - 필요한 요청: 아이템 구매, 간식 주기, 체크인 제출·수정, 탐색, 포획, 목표 완료, 카테고리 열기.
    - 키는 **사용자 + 메서드 + 경로** 단위로 구분하며 24시간 보관합니다.
    - 같은 키, 같은 본문이면 처음 결과(상태 코드 포함)를 그대로 돌려줍니다.
    - 같은 키, 다른 본문이면 422 `IDEMPOTENCY_KEY_REUSED`.
    - 같은 키의 첫 요청이 아직 처리 중이면 409 `REQUEST_IN_PROGRESS`. 앱은 잠시 뒤 같은 키로 다시 보냅니다.
- **요청 검사 순서**: ① 인증 → ② 온보딩 여부 → ③ 중복 요청 방지 키 조회(저장된 결과가 있으면 여기서 반환) → ④ 소유권 → ⑤ 업무 날짜 → ⑥ 값 형식 → ⑦ 비즈니스 규칙. 예를 들어 23:59에 보낸 요청이 타임아웃되어 00:01에 같은 키로 재전송되면, ③에서 처음 결과가 나가고 날짜 검사까지 가지 않습니다.
- **동시 차감**: 골드·볼·물약 차감은 조건부 갱신(`... WHERE gold >= ?`)이나 행 잠금으로 처리합니다. 키가 다른 요청 두 건이 동시에 들어와도 잔액이 음수가 되지 않으며, 부족하면 422를 돌려줍니다.
- **점수**
    - 습관 점수(`score`)는 0~25 정수(반올림)이며 **1점 = 1G**입니다. 공식은 아래 "4-1. 점수 공식"을 봅니다.
    - 성실도(`...Rate`)는 0~100 정수(%)입니다.
- **필드 이름**: JSON은 camelCase, 코드값은 DB 값을 그대로 씁니다 (예: `"WEEKLY"`, `"FIELD"`, `"D30"`).
- **PATCH 규칙**: 보내지 않은 필드는 바꾸지 않고, `null`을 보낸 필드는 지웁니다. 지울 수 없는 필드(필수 값)에 `null`을 보내면 400.
- **성공 응답**: 본문을 그대로 돌려줍니다. 돌려줄 것이 없으면 `204 No Content`.
- **에러 응답**: 아래 형식으로 통일합니다. 코드 전체 목록은 문서 끝의 "에러 코드 목록"을 봅니다.

```json
{ "code": "INVALID_REQUEST", "message": "입력값을 확인해 주세요.",
  "errors": [ { "field": "habits[0].dailyTarget", "reason": "0보다 커야 합니다." } ] }
```

`errors`는 선택 필드이며, 여러 필드가 동시에 틀릴 수 있는 400 응답(온보딩 일괄 제출 등)에서 씁니다.

상태 코드	쓰는 경우
400	값 형식·범위가 틀림 (빈 이름, 목표값 0 등)
401	토큰 없음·만료, 로그인 실패
403	본인 데이터가 아님, 온보딩 전
404	대상 없음
409	현재 상태에서 할 수 없음 (이미 완료한 목표, 오늘 이미 제출, 날짜 바뀜 등)
422	규칙 위반 (골드 부족, 슬롯 초과, 탐색 기회 없음 등)

한눈에 보기 (44개)

영역	메서드	경로	요청 이름
회원	POST	/auth/signup	이메일 회원가입
회원	POST	/auth/login	이메일 로그인
회원	POST	/auth/social/{provider}	소셜 로그인
회원	POST	/auth/refresh	토큰 재발급
회원	POST	/auth/logout	로그아웃
회원	PUT	/me/device-token	푸시 토큰 등록
회원	POST	/me/attendance	출석체크 (앱 접속 시 자동)
회원	DELETE	/me	회원 탈퇴
온보딩	GET	/categories	카테고리 목록 조회
온보딩	GET	/measure-types	측정 단위·추천 습관 조회
온보딩	POST	/onboarding	온보딩 완료 (일괄 제출)
메인	GET	/me/status	상단 바 조회
메인	GET	/me/today	오늘 요약 조회 (스크롤탭)
메인	GET	/monsters	내 몬스터 목록 (필드·가방)
메인	GET	/monsters/{id}	몬스터 상세 (팝업)
메인	POST	/monsters/{id}/affection	쓰다듬기·놀아주기
메인	POST	/monsters/{id}/feed	간식 주기
메인	GET	/monsters/{id}/line	말걸기
메인	PATCH	/monsters/{id}/location	가방에 넣기·꺼내기
메인	PUT	/me/partner	대화 상대 변경
메인	GET	/shop/items	상점 목록
메인	POST	/shop/purchases	아이템 구매 (볼 사기 포함)
메인	GET	/inventory	가방 아이템 조회

영역	메서드	경로	요청 이름
습관	GET	/habits	습관 목록 **(신규)**
습관	POST	/habits	습관 추가 **(신규)**
습관	POST	/habits/{habitId}/renew	기간 연장 **(신규)**
습관	POST	/habits/{habitId}/stop	습관 그만두기 **(신규)**
챗봇	GET	/checkins/today	체크인 준비 조회
챗봇	POST	/checkins/today	체크인 제출 (첫 제출)
챗봇	PUT	/checkins/today	체크인 수정 (다시 체크)
챗봇	POST	/checkins/today/dialogue	체크인 대사 요청
챗봇	POST	/explores	탐색하기·다시 탐색
챗봇	GET	/explores/pending	남은 등장 조회 **(신규)**
챗봇	POST	/explores/{exploreId}/capture	볼 던지기
통계	GET	/stats	통계 조회
도감	GET	/dex	도감 요약
도감	GET	/dex/categories/{code}	카테고리 몬스터 목록
도감	GET	/dex/species/{speciesId}	도감 몬스터 상세
도감	POST	/categories/{code}/unlock	카테고리 열기
목표	GET	/goals	목표 목록
목표	POST	/goals	목표 등록
목표	PATCH	/goals/{goalId}	목표 수정
목표	DELETE	/goals/{goalId}	목표 삭제
목표	POST	/goals/{goalId}/complete	목표 완료

1. 회원·설정

계정 규칙: 이메일 하나에 계정 하나입니다. 이메일 가입과 소셜 가입이 같은 이메일을 쓰더라도 자동으로 합치지 않습니다. 이메일 가입에 인증 절차가 없어서, 자동으로 합치면 남의 이메일로 먼저 가입해 두는 방식의 계정 탈취가 가능하기 때문입니다. 다른 방법으로 이미 가입된 이메일이면 409

`EMAIL_REGISTERED_WITH_OTHER_METHOD`와 함께 기존 가입 방법을 알려주고, 사용자는 그 방법으로 로그인합니다. 로그인한 상태에서 계정을 연결하는 기능은 확장 단계입니다.

```json
// 409 응답 예시
{ "code": "EMAIL_REGISTERED_WITH_OTHER_METHOD", "message": "카카오로 가입된 이메일입니다.", "pr
```

POST /auth/signup — 이메일 회원가입

```json
// 요청
{ "email": "user@example.com", "password": "********" }
// 응답 201
{ "accessToken": "eyJ...", "refreshToken": "eyJ...", "onboarded": false }
```

이메일은 소문자로 통일해 중복을 검사합니다. 이메일 가입으로 이미 있으면 409 `EMAIL_TAKEN`, 소셜로 가입된 이메일이면 409 `EMAIL_REGISTERED_WITH_OTHER_METHOD`.

POST /auth/login — 이메일 로그인

```json
// 요청
{ "email": "user@example.com", "password": "********" }
// 응답 200
{ "accessToken": "eyJ...", "refreshToken": "eyJ...", "onboarded": true }
```

이메일이나 비밀번호가 틀리면 401 `INVALID_CREDENTIALS`. `onboarded`가 false면 앱은 화면 1로 보냅니다.

POST /auth/social/{provider} — 소셜 로그인

`provider`: `kakao` | `google` (Apple 로그인은 이번 범위에서 제외)

```json
// 요청
{ "providerToken": "제공자가 준 인증 토큰" }
// 응답 200
{ "accessToken": "eyJ...", "refreshToken": "eyJ...", "isNewUser": true, "onboarded": false }
```

- 제공자 토큰 검증에 실패하면 401 `INVALID_PROVIDER_TOKEN`.
- 처음 로그인하는 사용자인데 제공자가 준 이메일이 다른 방법으로 이미 가입되어 있으면 409 `EMAIL_REGISTERED_WITH_OTHER_METHOD`.

- 카카오는 이메일 제공이 사용자 선택이므로, 이메일 없이도 가입됩니다.

POST /auth/refresh — 토큰 재발급

```
json

// 요청
{ "refreshToken": "eyJ..." }
// 응답 200 (리프레시 토큰도 새로 발급하고 이전 것은 폐기)
{ "accessToken": "eyJ...", "refreshToken": "eyJ..." }
```

리프레시 토큰이 만료·폐기되었으면 401 `INVALID_TOKEN`. 앱은 로그인 화면으로 보냅니다.

POST /auth/logout — 로그아웃

```
json

// 요청
{ "refreshToken": "eyJ..." }
// 응답 204
```

PUT /me/device-token — 푸시 토큰 등록

```
json

// 요청
{ "fcmToken": "...", "platform": "ANDROID" }
// 응답 204
```

같은 토큰이 다른 계정에 있으면 현재 계정으로 옮깁니다.

POST /me/attendance — 출석체크

앱이 실행되거나 백그라운드에서 돌아올 때 자동으로 부릅니다. 하루에 여러 번 불러도 됩니다.

```
json

// 응답 200
{ "date": "2026-10-12", "alreadyAttended": false, "currentStreak": 5, "longestStreak": 12 }
```

DELETE /me — 회원 탈퇴

```
json

// 응답 204
```

인증 정보·토큰·AI 대화를 지우고 습관명·목표 텍스트를 비웁니다. 탈퇴하면 토큰이 지워지므로, 이후 요청은 401 `INVALID_TOKEN`에서 막힙니다.

2. 온보딩

GET /categories — 카테고리 목록 조회

화면 2와 통계·도감의 카테고리 표시에 같이 씁니다.

```json
// 응답 200
[
  {
    "code": "ex", "name": "운동", "shortName": "운동", "element": "불꽃",
    "glyph": "🔥", "color": "#E8553D", "sortOrder": 1,
    "opened": false,
    "starter": { "speciesId": 1, "name": "몬스터 이름", "imageUrl": "https://..." }
  }
]
```

GET /measure-types?categories=ex,st — 측정 단위·추천 습관 조회

화면 3에서 고른 카테고리만 넘깁니다. 온보딩 이후 습관을 추가할 때도 씁니다.

```json
// 응답 200
[
  {
    "categoryCode": "ex",
    "measureTypes": [
      { "id": 1, "name": "운동 시간", "unit": "분", "scoringMode": "TARGET_RATIO",
        "defaultTarget": 30, "step": 5, "maxValue": 600 }
    ],
    "presets": [
      { "id": 1, "name": "아침 러닝", "measureTypeId": 1, "defaultTarget": 30 }
    ]
  }
]
```

기간 선택지(`D7`, `D30`, `D100`)는 고정값이라 앱에 둡니다.

POST /onboarding — 온보딩 완료 (일괄 제출)

화면 4의 시작 버튼에서 한 번만 부릅니다. 하나라도 틀리면 아무것도 저장하지 않고, 틀린 항목을 `errors`에 모두 담아 돌려줍니다.

```json
```

```
// 요청
{
  "categories": ["ex", "st"],
  "habits": [
    { "categoryCode": "ex", "measureTypeId": 1, "name": "걷기", "dailyTarget": 30, "periodCode"
  ],
  "goals": [
    { "goalType": "WEEKLY", "title": "헬스장 3번 가기" }
  ]
}
// 응답 201
{
  "starters": [
    { "userMonsterId": 10, "speciesId": 1, "categoryCode": "ex", "name": "몬스터 이름",
      "imageUrl": "https://...", "location": "FIELD" }
  ],
  "partnerMonsterId": 10,
  "habitIds": [100],
  "goalIds": [200]
}
```

검사	실패 시
이미 온보딩 완료	409 `ALREADY_ONBOARDED`
카테고리 1~3개, 중복 없음	400
습관 1~5개, 각 습관의 카테고리가 고른 카테고리 안에 있음, 측정 단위가 그 카테고리 것	400
습관 이름 1~30자, 목표값 > 0 그리고 입력 상한 이하	400
목표: 주간 0~3개, 월간 0~1개, 제목 1~50자	400

온보딩에서 등록한 습관은 오늘부터 시작합니다. 대화 상대는 운동 카테고리 몬스터, 운동을 고르지 않았으면 고른 카테고리 중 순서가 가장 앞선 카테고리의 몬스터입니다.

3. 상단 바·메인 화면

GET /me/status — 상단 바 조회

```
json
```

```
// 응답 200
{
  "businessDate": "2026-10-12",
  "currentStreak": 5,
  "gold": 340,
  "ballCount": 2,
  "checkedInToday": true,
  "exploreRemaining": 1,
  "exploreTotal": 3,
  "endedHabitCount": 0
}
```

- 체크인 전에는 `exploreRemaining`, `exploreTotal`이 0입니다.
- `endedHabitCount`는 기간을 채워 연장·그만두기 선택을 기다리는 습관 수입니다. 1 이상이면 앱이 선택을 안내합니다.

GET /me/today — 오늘 요약 조회 (스크롤탭)

오늘의 점수·성실도·골드는 보상 기준(첫 제출 값)입니다.

json

```
// 응답 200
{
  "businessDate": "2026-10-12",
  "checkedIn": true,
  "goldEarned": 353,
  "overallRate": 88,
  "habits": [
    { "habitId": 100, "name": "걷기", "categoryCode": "ex", "unit": "분",
      "dailyTarget": 30, "rewardedValue": 20, "rewardedScore": 13, "counted": true,
      "filledDays": 4, "periodDays": 30 }
  ]
}
```

체크인 전이면 `goldEarned`, `overallRate`, `rewardedValue`, `rewardedScore`, `counted`가 null입니다.

GET /monsters?location=FIELD — 내 몬스터 목록

`location`: `FIELD`(메인 필드) | `BAG`(가방 몬스터 탭) | 생략하면 전체(챗봇 대화 상대 선택).

json

```
// 응답 200
[
  { "userMonsterId": 10, "speciesId": 1, "name": "몬스터 이름", "categoryCode": "ex",
    "level": 7, "stage": 0, "imageUrl": "https://...", "displayHeight": 72,
    "facing": "FRONT", "location": "FIELD", "isPartner": true }
]
```

대화 상대는 가방에 있을 수 있습니다. 필드 표시와 챗봇 대화 상대는 별개입니다.

GET /monsters/{id} — 몬스터 상세 (팝업)

json

```
// 응답 200
{
  "userMonsterId": 10, "name": "몬스터 이름", "imageUrl": "https://...",
  "categoryCode": "ex", "stage": 0, "rarity": 2,
  "level": 7, "expInLevel": 40, "expToNextLevel": 125,
  "nextEvolutionLevel": 10,
  "affection": 6, "affectionMax": 10, "affectionTier": "MID",
  "todayActions": { "pet": true, "play": false },
  "location": "FIELD", "isPartner": true
}
```

최종 진화 단계면 `nextEvolutionLevel`은 null입니다.

POST /monsters/{id}/affection — 쓰다듬기·놀아주기

json

```
// 요청
{ "action": "PET" }   // PET 쓰다듬기 | PLAY 놀아주기
// 응답 200
{ "gain": 2, "affection": 8, "affectionTier": "HIGH" }
```

오늘 같은 행동을 이미 했으면 409 `ALREADY_DONE_TODAY`. 호감도가 최대면 `gain`은 0입니다.

POST /monsters/{id}/feed — 간식 주기

헤더 `Idempotency-Key` 필요.

json

```
// 요청
{ "itemCode": "POTION_M" }   // POTION_S | POTION_M | POTION_L
// 응답 200
{
  "expGained": 100, "levelBefore": 7, "levelAfter": 8,
  "stageBefore": 0, "stageAfter": 0, "evolved": false,
  "remainingQuantity": 2
}
```

물약이 없으면 422 `NO_ITEM`, 최대 레벨이면 422 `MAX_LEVEL`.

GET /monsters/{id}/line — 말걸기

```
json

// 응답 200
{ "affectionTier": "MID", "line": "오늘도 뛸 거지? 따라갈게." }
```

PATCH /monsters/{id}/location — 가방에 넣기·꺼내기

```
json

// 요청
{ "location": "BAG" }   // BAG 가방에 넣기 | FIELD 꺼내기
// 응답 200
{ "userMonsterId": 10, "location": "BAG", "fieldCount": 3 }
```

필드가 1마리일 때 넣기, 5마리일 때 꺼내기는 422 `FIELD_LIMIT`. 대화 상대도 가방에 넣을 수 있습니다.

PUT /me/partner — 대화 상대 변경

메인 팝업의 "대화 상대로"와 챗봇 상단 선택이 같이 씁니다.

```
json

// 요청
{ "userMonsterId": 12 }
// 응답 200
{ "partnerMonsterId": 12 }
```

GET /shop/items — 상점 목록

```
json
```

```
// 응답 200
{
  "gold": 340,
  "items": [
    { "code": "POTION_S", "name": "하급 경험치 물약", "shortName": "하급", "price": 20,
      "effectType": "MONSTER_EXP", "effectValue": 20 },
    { "code": "BALL", "name": "성실볼", "shortName": "성실볼", "price": 50,
      "effectType": "CAPTURE", "effectValue": 1 }
  ]
}
```

가격·효과 값은 shop_item 시드를 따릅니다.

POST /shop/purchases — 아이템 구매

헤더 `Idempotency-Key` 필요. 챗봇의 "볼 사기"도 이 요청을 씁니다.

json

```
// 요청
{ "itemCode": "BALL", "quantity": 1 }
// 응답 201
{ "purchaseId": 300, "totalPrice": 50, "gold": 290, "itemCode": "BALL", "quantity": 3 }
```

- 응답의 `quantity`는 구매 후 가방에 있는 개수입니다.
- 요청 `quantity`는 1~99. 범위를 벗어나면 400.
- 골드가 부족하면 422 `NOT_ENOUGH_GOLD`.

GET /inventory — 가방 아이템 조회

가방의 몬스터 탭은 `GET /monsters`를 씁니다.

json

```
// 응답 200
{
  "items": [
    { "itemCode": "POTION_S", "name": "하급 경험치 물약", "quantity": 2 },
    { "itemCode": "BALL", "name": "성실볼", "quantity": 3 }
  ]
}
```

4. 습관 (신규)

4-1. 점수 공식

측정 방식	점수	예시 (목표 30분)
목표 미달	20 × (기록 ÷ 목표)	20분 → 13
목표 달성	20	30분 → 20
목표 초과	20 + 10 × (기록 ÷ 목표 − 1), 최대 25	45분 이상 → 25
O/X	했으면 20, 못했으면 0	
범위(수면 시간)	목표 범위 안이면 20, 밖이면 0	

- 점수는 반올림한 정수이며, 공식의 수치는 `config.balance_config`(`score_*`)를 따릅니다.
- **달성률**(성실도 계산용) = min(점수 ÷ 20, 1) × 100. **반올림한 점수로 계산**합니다. 예: 13점 → 65%.
- **전체 성실도(`overallRate`)** = 점수 상위 3개 습관의 달성률 평균(반올림). 습관이 3개 미만이면 있는 습관만으로 평균을 냅니다.

4-2. 습관 상태와 기간

상태	뜻	체크인 대상
`ACTIVE`	기간 진행 중	O (시작일 ≤ 오늘)
`ENDED`	기간을 채워 연장·그만두기 선택을 기다림	X (다음 날부터)
`STOPPED`	그만둠. 통계·기록용으로만 남음	X (다음 날부터)

- **기간은 채운 날 수 기준**입니다. D7은 7일, D30은 30일, D100은 100일을 채우면 끝나며, 연속일 필요는 없습니다.
- **채운 날은 첫 제출에서 그 습관을 했다고(`done: true`) 제출한 날**입니다. "못했어"로 제출한 날과 체크인을 빠뜨린 날은 세지 않을 뿐, 기간이 끝나지는 않습니다. 오래 접속하지 않아도 채운 날은 사라지지 않습니다.
- 다시 체크(PUT)로 바꾼 값은 채운 날 수에 영향을 주지 않습니다.
- `filledDays`는 지금까지 채운 날 수, `periodDays`는 채워야 할 날 수(7/30/100)입니다. 종료일은 없습니다.
- 첫 제출로 `filledDays`가 `periodDays`에 닿으면 그 제출에서 기간 보너스를 주고 `ENDED`로 바뀝니다.
- 진행 중인 습관의 목표값은 바꿀 수 없습니다. 바꾸려면 기간을 채운 뒤 연장할 때 바꿉니다.
- 습관 개수 상한은 `ACTIVE` + `ENDED`를 합쳐 5개입니다.

- **시작일 규칙:** 습관을 추가하거나 연장하면, 오늘 체크인을 아직 제출하지 않았으면 오늘부터, 이미 제출했으면 내일부터 시작합니다. 제출한 체크인에 습관이 새로 끼어드는 일을 막기 위해서입니다.

GET /habits — 습관 목록

`status`: `ACTIVE` | `ENDED` | `STOPPED`, 쉼표로 여러 개. 생략하면 `ACTIVE,ENDED`.

```json
// 응답 200
[
  { "habitId": 100, "categoryCode": "ex", "name": "걷기", "measureTypeId": 1, "unit": "분",
    "dailyTarget": 30, "periodCode": "D30", "startDate": "2026-10-09",
    "filledDays": 4, "periodDays": 30, "status": "ACTIVE" }
]
```

POST /habits — 습관 추가

온보딩 이후 습관을 추가하거나, 카테고리를 새로 연 뒤 그 카테고리의 습관을 등록할 때 씁니다.

```json
// 요청
{ "categoryCode": "md", "measureTypeId": 12, "name": "10분 명상", "dailyTarget": 10, "periodCod
// 응답 201: GET /habits의 항목과 같은 모양
```

검사	실패 시
이름 1~30자, 목표값 > 0 그리고 입력 상한 이하, 측정 단위가 그 카테고리 것	400
열지 않은 카테고리	422 `CATEGORY_LOCKED`
`ACTIVE` + `ENDED`가 이미 5개	422 `HABIT_LIMIT`

POST /habits/{habitId}/renew — 기간 연장

`ENDED` 습관을 새 기간으로 다시 시작합니다. 목표값을 바꿀 수 있습니다.

```json
// 요청: dailyTarget을 생략하면 이전 값 유지
{ "periodCode": "D30", "dailyTarget": 40 }
// 응답 200: GET /habits의 항목과 같은 모양 (status: ACTIVE, filledDays: 0부터 다시 셈)
```

검사	실패 시
`ENDED`가 아님	409 `HABIT_NOT_ENDED`
목표값 범위	400

POST /habits/{habitId}/stop — 습관 그만두기

```json
// 응답 200
{ "habitId": 100, "status": "STOPPED" }
```

- `ACTIVE`와 `ENDED` 모두 그만둘 수 있습니다. 진행 중에 그만두면 채운 날과 쌓은 보상은 남지만 기간 보너스는 없습니다.
- 이미 그만둔 습관이면 409 `HABIT_ALREADY_STOPPED`.
- 오늘 체크인을 제출한 뒤 그만두면, 그날의 다시 체크(PUT)에는 계속 포함되고 다음 날부터 빠집니다.
- 앱은 그만두기 전에 확인 창을 띄웁니다.

5. 챗봇 — 체크인·탐색·포획

GET /checkins/today — 체크인 준비 조회

```json
// 응답 200
{
  "businessDate": "2026-10-12",
  "submitted": false,
  "revision": 0,
  "partner": { "userMonsterId": 10, "name": "몬스터 이름", "imageUrl": "https://...", "affectio
  "tabs": [
    {
      "categoryCode": "ex", "categoryName": "운동",
      "habits": [
        { "habitId": 100, "name": "걷기", "unit": "분", "scoringMode": "TARGET_RATIO",
          "options": [10, 20, 30, 40, 60], "previousValue": null,
          "filledDays": 4, "periodDays": 30 }
      ]
    }
  ]
}
```

- 제출 전: `ACTIVE`이고 시작일 ≤ 오늘인 습관이 들어갑니다. 대상이 없으면 `tabs`는 빈 배열입니다.
- 제출 후: 첫 제출에 들어간 습관이 들어갑니다(다시 체크 대상).
- `options`는 등록한 단위에서 고를 수 있는 값입니다. O·X 습관은 `[1]`(O)이며 "못했어"가 X입니다.
- 이미 제출했으면 `previousValue`에 마지막 값이 들어 있어 다시 체크할 때 미리 채웁니다.

POST /checkins/today — 체크인 제출 (첫 제출)

헤더 `Idempotency-Key` 필요. 탭은 화면 구분일 뿐이며, **오늘 체크인 대상인 습관 전체를 한 번에** 보냅니다.

```
json
```

```
// 요청
{
  "businessDate": "2026-10-12",
  "answers": [
    { "habitId": 100, "done": true, "value": 20 },
    { "habitId": 101, "done": false },
    { "habitId": 102, "done": true, "value": 60 },
    { "habitId": 103, "done": true, "value": 2 }
  ]
}
// 응답 201
{
  "revision": 1,
  "habits": [
    { "habitId": 100, "score": 13, "counted": true },
    { "habitId": 101, "score": 0, "counted": false },
    { "habitId": 102, "score": 20, "counted": true },
    { "habitId": 103, "score": 20, "counted": true }
  ],
  "overallRate": 88,
  "goldEarned": 353,
  "goldBreakdown": { "score": 53, "levelUp": 20, "periodBonus": 280 },
  "categoryExp": [
    { "categoryCode": "ex", "expGained": 13, "levelBefore": 3, "levelAfter": 4, "levelUpGold":
    { "categoryCode": "st", "expGained": 20, "levelBefore": 8, "levelAfter": 8, "levelUpGold":
    { "categoryCode": "fd", "expGained": 20, "levelBefore": 5, "levelAfter": 5, "levelUpGold":
  ],
  "periodBonuses": [
    { "habitId": 102, "periodCode": "D30", "multiplier": 1.5, "periodScore": 560, "bonusGold":
  ],
  "unlockedSpecies": [],
  "exploreTotal": 3,
  "gold": 693
}
```

보상 규칙

- **점수:** 습관마다 0~25 (4-1 공식). `done: false`는 값 0, 점수 0으로 저장합니다.
- **반영 습관(`counted`):** 점수 상위 3개 습관. 골드·카테고리 경험치·기간 보너스는 반영 습관에만 적용합니다. 점수가 같으면 등록 순서가 빠른 습관을 고릅니다.
- **골드:** 반영 습관 점수의 합 (1점 = 1G). 위 예시는 13 + 20 + 20 = 53G.
- **카테고리 경험치:** 반영 습관의 점수가 각 습관의 카테고리 경험치로 들어갑니다. 레벨업 필요 경험치는 10 × 현재 레벨, 최대 레벨 20, 레벨업마다 20G. 위 예시의 ex는 Lv.3에서 경험치 20

을 가지고 있었고, 13을 받아 33이 되어 30을 쓰고 Lv.4(남은 경험치 3)가 됩니다.

- **종 해금**: 레벨업으로 탐색에 새로 나오게 된 종이 있으면 `unlockedSpecies`에 `{ speciesId, categoryCode }`로 담깁니다.
- **기간 보너스**: 이번 제출로 습관의 `filledDays`가 `periodDays`에 닿으면, 그 기간에 그 습관이 **반영 습관으로 받은 점수의 합**(`periodScore`, 첫 제출 기준)에 `(배수 − 1)`을 곱한 만큼 골드를 줍니다. 배수는 D7 1.2, D30 1.5, D100 2.0입니다. 위 예시는 560 × 0.5 = 280G. 이 습관은 `ENDED`가 됩니다.
- `goldEarned` = 점수 골드 + 레벨업 골드 + 기간 보너스 골드. 구성은 `goldBreakdown`에 나눠 담습니다.
- `overallRate`: 4-1 공식. 위 예시는 (100 + 100 + 65) ÷ 3 = 88.
- **탐색 횟수**: 첫 제출이 전부 "못했어"(`done: false`)이면 `exploreTotal` 1, 그 외에는 3.
- 보상·경험치·채운 날·탐색 횟수는 모두 첫 제출 기준이며 다시 체크해도 바뀌지 않습니다.

알려진 한계: 자기보고 방식이라 첫 제출을 부풀리거나, 가장 작은 값을 "했어"로 내 탐색 3회를 받는 것을 막을 수 없습니다. 다시 체크로 값을 낮춰도 보상이 유지되는 것은 정직한 정정을 벌하지 않기 위한 의도된 선택입니다. 수행 여부를 검증하는 시스템은 확장 단계에서 추가합니다.

검사	실패 시
`businessDate`가 서버 오늘 날짜와 다름	409 `BUSINESS_DATE_CHANGED`
오늘 이미 제출	409 `ALREADY_SUBMITTED` (수정은 PUT)
오늘 체크인 대상 습관이 없음	409 `NO_ACTIVE_HABIT` (앱은 습관 추가·연장을 안내)
대상 습관을 모두 빠짐없이 보냈는지, 본인 습관인지	400
`done: true`면 `value`가 그 습관의 선택지 중 하나	400

PUT /checkins/today — 체크인 수정 (다시 체크)

헤더 `Idempotency-Key` 필요. 요청 본문은 POST와 같으며, 보낼 습관은 **오늘 첫 제출에 들어간 습관 전체**입니다. 첫 제출 뒤에 기간을 채웠거나 그만둔 습관도 포함합니다.

```
json
```

```
// 응답 200: 보상은 변하지 않음. 습관 전체를 돌려줌
{
  "revision": 2,
  "habits": [
    { "habitId": 100, "value": 40, "score": 23 },
    { "habitId": 101, "value": 0, "score": 0 },
    { "habitId": 102, "value": 60, "score": 20 },
    { "habitId": 103, "value": 2, "score": 20 }
  ],
  "overallRate": 100
}
```

- 오늘 제출이 없으면 404 `CHECKIN_NOT_FOUND`. 어제 이전 기록은 고칠 수 없습니다.
- 여기의 점수와 `overallRate`는 다시 체크한 최신 값이며, 통계 그래프에 쓰입니다.

POST /checkins/today/dialogue — 체크인 대사 요청

제출·수정 응답을 받은 뒤 부릅니다. 대사가 늦어도 제출 결과에는 영향이 없습니다.

```
json

// 응답 200
{ "revision": 1, "line": "오늘 20분이나 걸었구나!", "source": "LLM" }   // source: LLM | TEMPLA
```

- 대사는 오늘 체크인의 최신 `revision`당 한 번만 만듭니다. 같은 `revision`에 다시 부르면 저장된 대사를 그대로 돌려줍니다.
- AI가 실패하거나 3초를 넘기면 템플릿 대사를 `source: "TEMPLATE"`로 돌려줍니다.
- 오늘 제출이 없으면 409 `CHECKIN_REQUIRED`.

POST /explores — 탐색하기·다시 탐색

헤더 `Idempotency-Key` 필요.

등장 규칙

- 탐색 결과는 셋입니다: 꽝(`MISS`), 가진 종(`DUPLICATE`), 안 가진 종(`ENCOUNTER`).
- 꽝 확률은 35%이며 설정값(`explore_miss_rate`)입니다.
- 등장 후보는 **연 카테고리의 종 중 해금 레벨 ≤ 카테고리 레벨**인 종입니다.
- 후보마다 가중치 = 희귀도 가중치 × (안 가진 종이면 가중치 배수). 희귀할수록 적게 나오고, 안 가진 종이 더 자주 나옵니다. 가중치는 설정값입니다.
- **가진 종**이 나오면 볼 없이 바로 골드(15G)를 받고 그 탐색은 끝납니다.
- **안 가진 종**이 나오면 볼을 던져 포획을 시도할 수 있습니다.

```
json
```

```
// 요청
{ "businessDate": "2026-10-12" }
// 응답 201: 안 가진 종 등장
{
  "exploreId": 500,
  "result": "ENCOUNTER",
  "species": { "speciesId": 7, "name": "몬스터 이름", "imageUrl": "https://...",
               "categoryCode": "st", "rarity": 2, "captureRate": 0.6 },
  "goldAwarded": 0,
  "gold": 340,
  "skippedExploreId": null,
  "exploreRemaining": 2
}
// 응답 201: 가진 종 등장 (자동 골드)
{
  "exploreId": 501,
  "result": "DUPLICATE",
  "species": { "speciesId": 1, "name": "몬스터 이름", "imageUrl": "https://...",
               "categoryCode": "ex", "rarity": 3, "captureRate": null },
  "goldAwarded": 15,
  "gold": 355,
  "skippedExploreId": null,
  "exploreRemaining": 1
}
// 응답 201: 꽝
{ "exploreId": 502, "result": "MISS", "species": null, "goldAwarded": 0, "gold": 355,
  "skippedExploreId": null, "exploreRemaining": 0 }
```

처리하지 않은 등장이 있는 상태에서 다시 탐색하면 그 등장은 사라지고 `skippedExploreId`에 번호가 담깁니다.

검사	실패 시
`businessDate`가 서버 오늘 날짜와 다름	409 `BUSINESS_DATE_CHANGED`
오늘 체크인 제출 전	409 `CHECKIN_REQUIRED`
남은 기회 0	422 `NO_EXPLORE_LEFT`

GET /explores/pending — 남은 등장 조회 (신규)

챗봇 화면에 들어올 때 부릅니다. "그만"을 눌렀거나 앱이 꺼져서 처리되지 않은 오늘의 등장(`ENCOUNTER`)을 돌려줍니다.

```
json
```

```
// 응답 200: 남은 등장 있음 (POST /explores의 ENCOUNTER 응답과 같은 모양)
// 응답 204: 남은 등장 없음
```

등장은 그날이 끝날 때(다음 날 00시)까지 남고, 그 뒤에는 사라집니다.

POST /explores/{exploreId}/capture — 볼 던지기

헤더 `Idempotency-Key` 필요. 볼 1개를 쓰고, 종의 희귀도별 포획률로 성공 여부가 정해집니다. **실패하면 몬스터는 도망가고 그 등장은 끝납니다.**

json

```
// 요청
{ "businessDate": "2026-10-12" }
// 응답 200: 포획 성공
{
  "outcome": "CAPTURED",
  "monster": { "userMonsterId": 15, "name": "몬스터 이름", "imageUrl": "https://...", "location
  "ballCount": 1,
  "dex": { "dexCount": 5, "newSlotAvailable": true }
}
// 응답 200: 포획 실패
{ "outcome": "ESCAPED", "monster": null, "ballCount": 1, "dex": { "dexCount": 4, "newSlotAvaila
```

- 포획률은 희귀도 1/2/3이 각각 80%/60%/40%이며 설정값입니다. 등장 응답의 `captureRate`로 미리 보여줍니다.
- 필드가 5마리면 새 몬스터는 가방(`BAG`)으로 갑니다.
- `newSlotAvailable`이 true면 도감 슬롯 진행바가 차서 카테고리를 열 수 있습니다.
- "그만" 버튼은 서버를 부르지 않습니다.

검사	실패 시
`businessDate`가 서버 오늘 날짜와 다름	409 `BUSINESS_DATE_CHANGED`
본인의 탐색이 아님	403 `FORBIDDEN`
등장(`ENCOUNTER`)이 아니거나, 이미 처리됐거나, 다시 탐색·날짜 변경으로 사라진 등장	409 `ENCOUNTER_CLOSED`
성실볼 0개	422 `NO_BALL` (앱은 볼 사기 버튼을 보여줌)

6. 통계

GET /stats — 통계 조회

```json
// 응답 200
{
  "attendance": {
    "currentStreak": 5, "longestStreak": 12,
    "last14Days": [ { "date": "2026-09-29", "attended": true } ]
  },
  "sincerity": {
    "last7Days": [ { "date": "2026-10-06", "dayOfWeek": "MON", "rate": 80 } ],
    "averageRate": 72
  },
  "categories": [
    { "code": "ex", "name": "운동", "opened": true, "level": 4, "maxLevel": 20,
      "expInLevel": 3, "expToNextLevel": 40, "levelUpGold": 20 },
    { "code": "md", "name": "명상", "opened": false }
  ]
}
```

- `last14Days`는 오래된 날부터 14개, `last7Days`는 7개입니다.
- 성실도 그래프의 `rate`는 다시 체크한 최신 `overallRate`입니다. 체크인하지 않은 날은 null. `averageRate`는 null을 뺀 날들의 평균입니다.
- 카테고리 레벨은 보상 기준이라 다시 체크해도 바뀌지 않습니다. 최대 레벨이면 `expToNextLevel`은 null입니다. 잠긴 카테고리는 레벨 필드가 없습니다.

7. 도감

GET /dex — 도감 요약

```json
// 응답 200
{
  "slot": { "dexCount": 7, "nextMilestone": 10, "availableTickets": 1 },
  "categories": [
    { "code": "ex", "name": "운동", "opened": true, "ownedCount": 4, "totalCount": 7 }
  ]
}
```

슬롯 진행바는 `dexCount / nextMilestone`입니다. `availableTickets`가 1 이상이면 열 수 있는 카테고리가 있습니다.

GET /dex/categories/{code} — 카테고리 몬스터 목록

```json
// 응답 200
{
  "code": "ex", "ownedCount": 4, "totalCount": 7,
  "monsters": [
    { "dexNo": 1, "owned": true, "speciesId": 1, "name": "몬스터 이름",
      "imageUrl": "https://...", "level": 7 },
    { "dexNo": 2, "owned": false, "unlockLevel": 3, "unlocked": true }
  ]
}
```

가지지 않은 종은 번호와 해금 정보만 보내 "?"로 표시합니다. `unlocked`가 false면 앱은 "카테고리 Lv.{unlockLevel}에 등장"을 보여줄 수 있습니다.

GET /dex/species/{speciesId} — 도감 몬스터 상세

```json
// 응답 200
{
  "dexNo": 1, "name": "몬스터 이름", "imageUrl": "https://...",
  "element": "불꽃", "rarity": 3, "owned": true,
  "level": 7, "stage": 0, "description": "설명 문구"
}
```

가지지 않은 종이면 404 `NOT_DISCOVERED`. `element`는 카테고리 속성이며, "타입"으로 표시합니다.

POST /categories/{code}/unlock — 카테고리 열기

헤더 `Idempotency-Key` 필요.

```json
// 응답 201
{
  "categoryCode": "md",
  "starter": { "userMonsterId": 20, "name": "몬스터 이름", "imageUrl": "https://...", "location
  "availableTickets": 0
}
```

검사	실패 시
열 수 있는 슬롯 없음	422 `NO_TICKET`
이미 연 카테고리	409 `ALREADY_OPENED`

새 카테고리는 Lv.1부터 시작하며, 초기 몬스터는 필드가 5마리면 가방으로 갑니다. 카테고리를 연 뒤 앱은 그 카테고리의 습관 추가(`POST /habits`)로 안내합니다.

8. 목표

주간 목표는 월요일 00시, 월간 목표는 1일 00시에 새 기간이 시작됩니다.

GET /goals — 목표 목록

```json
// 응답 200
{
  "weekly": {
    "periodStart": "2026-10-12", "slotsTotal": 3, "slotsUsed": 1, "rewardGold": 40,
    "goals": [ { "goalId": 200, "title": "헬스장 3번 가기", "detail": null, "status": "ACTIVE"
  },
  "monthly": {
    "periodStart": "2026-10-01", "slotsTotal": 1, "slotsUsed": 0, "rewardGold": 150,
    "goals": []
  }
}
```

이번 주와 이번 달의 목표만 돌려줍니다. 삭제한 목표는 빠지고 완료한 목표는 `COMPLETED`로 남습니다.

POST /goals — 목표 등록

```json
// 요청
{ "goalType": "WEEKLY", "title": "헬스장 3번 가기", "detail": "월 · 수 · 금" }
// 응답 201
{ "goalId": 201, "goalType": "WEEKLY", "periodStart": "2026-10-12",
  "title": "헬스장 3번 가기", "detail": "월 · 수 · 금", "status": "ACTIVE" }
```

제목 1~50자, 상세 0~500자. 슬롯이 다 찼으면 422 `GOAL_SLOT_FULL`.

PATCH /goals/{goalId} — 목표 수정

```json
```

```
// 요청: 바꿀 필드만. 상세를 지우려면 "detail": null
{ "title": "헬스장 4번 가기", "detail": null }
// 응답 200: 등록 응답과 같은 모양
```

`title`은 지울 수 없으므로 `null`이면 400. `goalType`은 바꿀 수 없습니다.

DELETE /goals/{goalId} — 목표 삭제

```
json

// 응답 204
```

POST /goals/{goalId}/complete — 목표 완료

헤더 `Idempotency-Key` 필요.

```
json

// 응답 200
{ "goalId": 200, "status": "COMPLETED", "goldAwarded": 40, "gold": 507 }
```

- 수정·삭제·완료 모두 완료한 목표에는 409 `GOAL_COMPLETED`. 완료는 되돌릴 수 없습니다.
- 지난 기간의 목표는 수정·삭제·완료할 수 없으며 409 `GOAL_PERIOD_ENDED`.

에러 코드 목록

상태	코드	상황
400	`INVALID_REQUEST`	값 형식·범위가 틀림 (필드별 이유는 `errors`)
401	`INVALID_TOKEN`	토큰 없음·형식 오류·폐기됨
401	`TOKEN_EXPIRED`	액세스 토큰 만료 (재발급 시도)
401	`INVALID_CREDENTIALS`	이메일·비밀번호 불일치
401	`INVALID_PROVIDER_TOKEN`	소셜 제공자 토큰 검증 실패
403	`FORBIDDEN`	본인 데이터가 아님
403	`ONBOARDING_REQUIRED`	온보딩 전 사용자
404	`NOT_FOUND`	대상 없음 (일반)
404	`NOT_DISCOVERED`	도감에서 가지지 않은 종
404	`CHECKIN_NOT_FOUND`	오늘 제출 없이 수정
409	`EMAIL_TAKEN`	이메일 가입으로 이미 가입된 이메일
409	`EMAIL_REGISTERED_WITH_OTHER_METHOD`	다른 가입 방법으로 가입된 이메일 (`provider` 포함)
409	`ALREADY_ONBOARDED`	이미 온보딩 완료
409	`ALREADY_DONE_TODAY`	오늘 이미 쓰다듬기·놀아주기
409	`ALREADY_SUBMITTED`	오늘 체크인 이미 제출
409	`NO_ACTIVE_HABIT`	체크인 대상 습관 없음
409	`CHECKIN_REQUIRED`	체크인 전 탐색·대사 요청
409	`ENCOUNTER_CLOSED`	포획할 수 없는 등장
409	`BUSINESS_DATE_CHANGED`	요청의 날짜가 서버 오늘과 다름
409	`HABIT_NOT_ENDED`	기간을 채우지 않은 습관 연장
409	`HABIT_ALREADY_STOPPED`	이미 그만둔 습관
409	`ALREADY_OPENED`	이미 연 카테고리

상태	코드	상황
409	`GOAL_COMPLETED`	완료한 목표 변경
409	`GOAL_PERIOD_ENDED`	지난 기간 목표 변경
409	`REQUEST_IN_PROGRESS`	같은 중복 방지 키의 요청 처리 중
422	`IDEMPOTENCY_KEY_REUSED`	같은 키에 다른 본문
422	`NOT_ENOUGH_GOLD`	골드 부족
422	`NO_ITEM`	물약 없음
422	`NO_BALL`	성실볼 없음
422	`MAX_LEVEL`	몬스터 최대 레벨
422	`FIELD_LIMIT`	필드 몬스터 수 제한
422	`NO_EXPLORE_LEFT`	탐색 기회 없음
422	`NO_TICKET`	카테고리 해금권 없음
422	`CATEGORY_LOCKED`	열지 않은 카테고리에 습관 추가
422	`HABIT_LIMIT`	습관 5개 초과
422	`GOAL_SLOT_FULL`	목표 슬롯 초과

DB·시드에 반영할 것

설정값 (`config.balance_config`)

- [ ] `day_boundary_hour`: 4 → **0**
- [ ] `category_max_level`: 7 → **20**
- [ ] `period_completion_rate` **삭제** (채운 날 수 방식으로 대체)
- [ ] `period_reset_days` **삭제** (오래 접속하지 않아도 채운 날 유지)
- [ ] `capture_rate` 유지 (희귀도별 0.8/0.6/0.4)
- [ ] 탐색 가중치 키 추가: 희귀도별 등장 가중치, 안 가진 종 가중치 배수 (시작값은 튜닝 대상)
- [ ] `duplicate_gold` (15) 설명 변경: 가진 종 등장 시 볼 없이 지급
- [ ] `explore_miss_rate`: **0.35** (베타에서 튜닝)

마스터 데이터

- [ ] 종 `unlock_level` 분산: 카테고리 7종 기준 1/3/6/9/12/15/18

스키마

- [ ] **출석 기록 테이블**: 날짜별 출석. 연속 일수(`habit_user.current_streak`)를 체크인 기준에서 출석 기준으로 변경
- [ ] **습관 상태·기간 기록**: `ACTIVE` / `ENDED` / `STOPPED` 상태, 연장마다 기간(시작일·기간 코드·목표값·채운 날 수)과 기간 중 반영 점수 합. 종료일 컬럼은 필요 없음
- [ ] **탐색 결과**: `DUPLICATE` 결과 추가(자동 골드), 포획 결과는 `CAPTURED` / `ESCAPED`
- [ ] **체크인 대사 저장**: 체크인 `revision`별 대사와 출처(LLM·TEMPLATE)
- [ ] **중복 요청 방지 키 저장**: 사용자, 메서드, 경로, 키, 본문 해시, 처리 상태, 결과(상태 코드·본문), 만료 시각
- [ ] **이메일 고유 제약**: 가입 방법과 관계없이 이메일은 한 계정에만 (소셜 이메일이 없으면 NULL 허용)
- [ ] **차감 처리**: 골드·볼·물약은 조건부 UPDATE 또는 행 잠금. `CHECK(balance >= 0)` 위반은 422로 변환

화면

- [ ] 습관 기간 문구를 7일/30일/100일로 맞춤

정해야 할 규칙

현재 남은 항목 없음. 탐색 가중치·꽝 확률은 베타에서 튜닝합니다. 검토 후 보류한 항목(이메일 선점, 초반 등장 후보)은 「API 명세 보류 항목」 문서를 봅니다.